

PHPLogger

Logging strutturato per PHP 7.4+ · Michael Lavigna, 18 settembre 2026

PHPLogger produce log applicativi strutturati: una riga JSON per evento, su uno schema fisso e versionato. Quello che la distingue non è il formato, ormai comune, ma il modo in cui protegge i dati sensibili: le strutture libere vengono **scartate se non esplicitamente permesse** in configurazione, invece di essere ispezionate a posteriori sperando di riconoscere un segreto. Detto altrimenti: **Monolog risolve il problema di avere dei log, PHPLogger quello di cosa c'è dentro**. Ha senso per noi solo se la seconda domanda ha una rilevanza di conformità o di sicurezza.

4.200righe di codice
di produzione**600**test automatici
94% di copertura**0**dipendenze
a runtime**~20 µs**per record
~50.000 rec/s**~630 B**dimensione tipica
di un record

Punti di forza e limiti

A favore

- **Nessuna dipendenza a runtime.** Nessuna superficie di supply chain, nessun conflitto di versioni.
- **Protezione per costruzione,** non affidata alla disciplina di chi scrive la singola chiamata.
- **Adattatori pronti e testati** per PDO, cURL e i gestori di errore, identici su 7.4.3, 8.1 e 8.3.
- **Schema di uscita bloccato dai test:** chi costruisce cruscotti non se lo vede cambiare sotto i piedi.
- **Il logging non può fermare l'applicazione:** guardia sull'intera catena, circuit breaker, percorso di emergenza.
- **600 test, 94% di copertura,** suite che gira sul runtime minimo dichiarato.

Contro

- **Distribuzione per copia (alta).** Niente Packagist: aggiornamenti manuali e copie che divergono fra servizi. È il rischio operativo maggiore.
- **Nessun tag di versione (media),** e i nomi delle interfacce sono cambiati di recente.
- **Configurazione da compilare (media).** Finché la lista dei campi permessi è vuota, parametri e contesto escono vuoti.
- **Scritture sincrone (media)** sul percorso della richiesta: nessun accodamento né invio asincrono.
- **Un solo autore (media).** In caso di adozione la manutenzione ricadrebbe interamente su di noi.
- **Un solo nome evento per connessione (bassa)** nell'adattatore PDO.

Quanto costa, misurato

Operazione	µs per record	Record al secondo
Query di database	19,94	50.156
Eccezione con stack trace	15,37	65.042
Messaggio applicativo semplice	11,91	83.957
Sovraccarico dell'adattatore PDO, per query	17,74	-

Misurati su PHP 7.4.3, il runtime più lento che la libreria supporta, con scrittura reale a destinazione. **In pratica:** una richiesta che registra 100 query di database paga circa 2 millisecondi, quasi tutti di CPU per la sanificazione. Sul sovraccarico per query conta il numero assoluto, non quello relativo: su una query MySQL da 1 millisecondo sono **+1,8%**, su una da 200 µs sono **+8,9%**. Dove pesa troppo, il campionamento è configurabile per tipo di log e non scarta mai i fallimenti né i record di livello error o superiore.

Raccomandazione

Non una decisione di adozione, ma una prova limitata

Rispetto alla valutazione iniziale la voce di costo principale - scrivere gli adattatori verso PDO, cURL e i gestori di errore - è **stata chiusa**: ci sono, con i propri test. Quello che resta da misurare è il lavoro di configurazione e, soprattutto, se i log che ne escono sono davvero più utili di quelli attuali.

Propongo una prova su **un solo servizio non critico**: collegare l'adattatore PDO, compilare la lista dei campi permessi e tenerlo in esercizio per qualche settimana. Il rischio è contenuto: per costruzione un guasto del logger non ferma l'applicazione.

Da tenere presente, ma è una discussione separata: PHP 7.4 è fuori supporto dal novembre 2022. La libreria lo supporta deliberatamente, il che è un vantaggio se il nostro parco applicativo è ancora lì. Ma se lo è, il rischio rilevante per la sicurezza non è il logger: è l'interprete.